

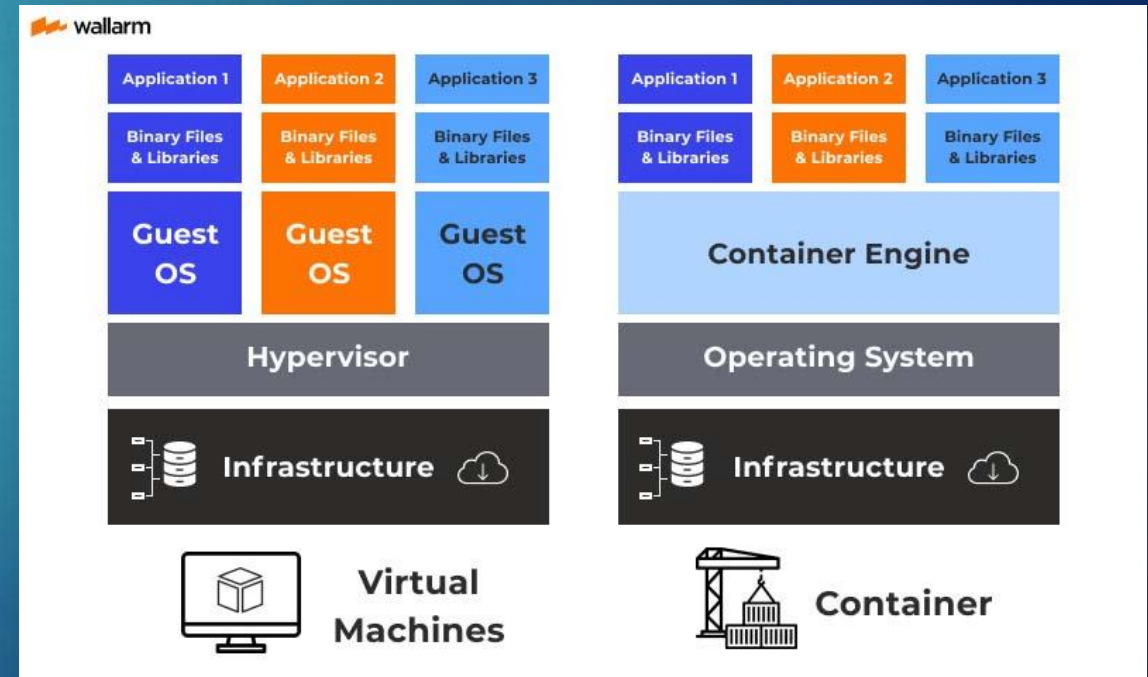
Project Proposal Presentation

1

- ▶ Upload to **Assignments > Term Project > Term Project Proposal**
 - ▶ 5% of your project grade
 - ▶ Teams are required to present a 5-10 minute proposal to introduce their project to the class. The presentation should be in PPT or PDF format and include at least 5 slides covering the following:
 - ▶ **Team Name and Project Overview:** Briefly introduce the team and outline the project objectives.
 - ▶ **AWS Services Overview:** List the AWS services you plan to use, with a brief description of how each will support project goals.
 - ▶ **High-Level Data Flow Diagram:** Present a diagram showing the interaction of AWS services and the primary data flow within the project.
 - ▶ Check out <https://app.diagrams.net> for drawing diagrams
 - ▶ It's free and offers lots of AWS shapes
 - ▶ More details can be found at **Content > Project > 514 Term Project**

- ▶ What is the difference between a Docker image and a Docker container?
 - A. Images run databases while containers run UI
 - B. An image is a running process; a container is a static file
 - C. A container is a running instance of a Docker image ☒
 - D. Containers create images when executed
- ▶ What is one benefit of separating services into different containers?
 - A. It reduces the number of servers required
 - B. It removes the need for a database
 - C. It allows better scalability and fault isolation ☒
 - D. It ensures a single point of failure
- ▶ Containers can help avoid vendor lock-in by enabling portability between cloud providers
 - ▶ True ☒
 - ▶ False

- ▶ Containers and virtual machines (VMs) offer ways to virtualize computing resources, but they differ in their architecture and use cases
 - ▶ Virtual Machines run on hypervisors, each with its own operating system, binaries, and libraries, which can make them resource-heavy
 - ▶ Containers share the host operating system, making them lightweight, faster to start, and more resource-efficient
 - ▶ Containers use fewer resources and are ideal for deploying microservices or lightweight applications
 - ▶ Containers can run on any platform, improving consistency across development, testing, and production environments

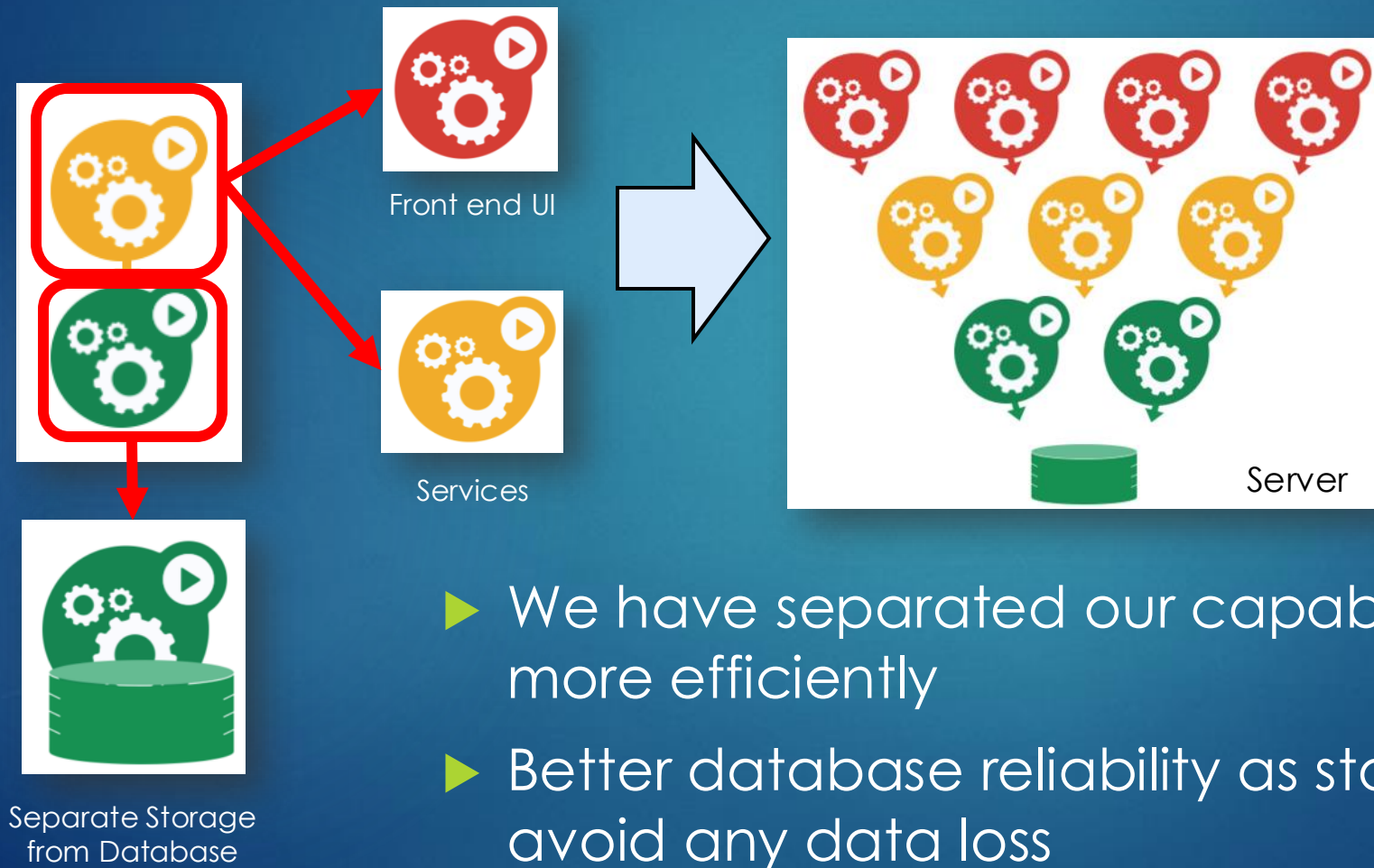


- ▶ Containers offer significant benefits for cloud-based deployments, including scalability, efficiency, and application isolation
- ▶ Other benefits include:
 - ▶ **Fast Deployment:** Containers start in milliseconds, making them ideal for dynamic scaling and short-lived tasks
 - ▶ **Cost Efficiency:** Sharing the host OS kernel reduces overhead, allowing more applications to run on the same hardware
 - ▶ **Isolation:** Each container runs independently, preventing conflicts between dependencies or updates
 - ▶ **Scalability:** Containers can be easily scaled horizontally to meet demand without downtime



Containerization Activity

- Split up the front-end services to separate responsibilities

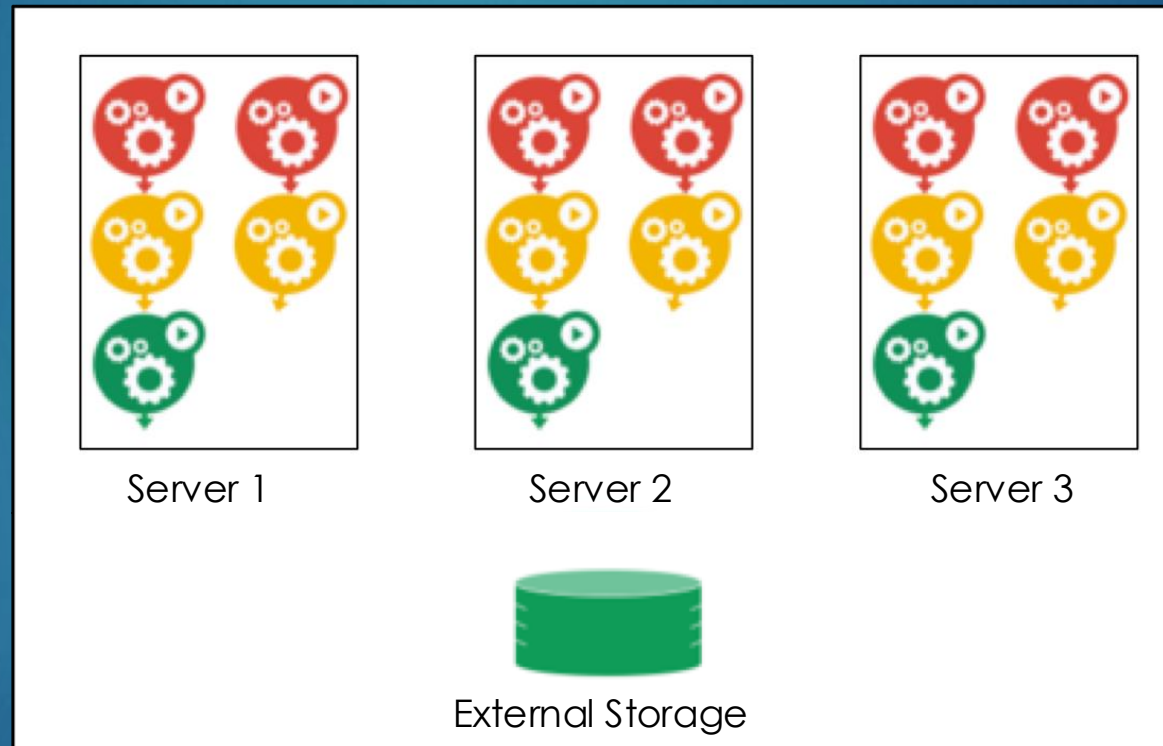


If this is running on one server, what happens if it runs out of resources or crashes?

- We have separated our capabilities and can scale more efficiently
- Better database reliability as storage is detached to avoid any data loss

Containerization – Scalable and Resilient

- We now have a much more scalable and resilient system using containers!



- So how can we manage all the containers running on all the servers?
- This is where Container Orchestration can help

Container Orchestration

Engineering Cloud Software Systems

Department of Software Engineering
Rochester Institute of Technology



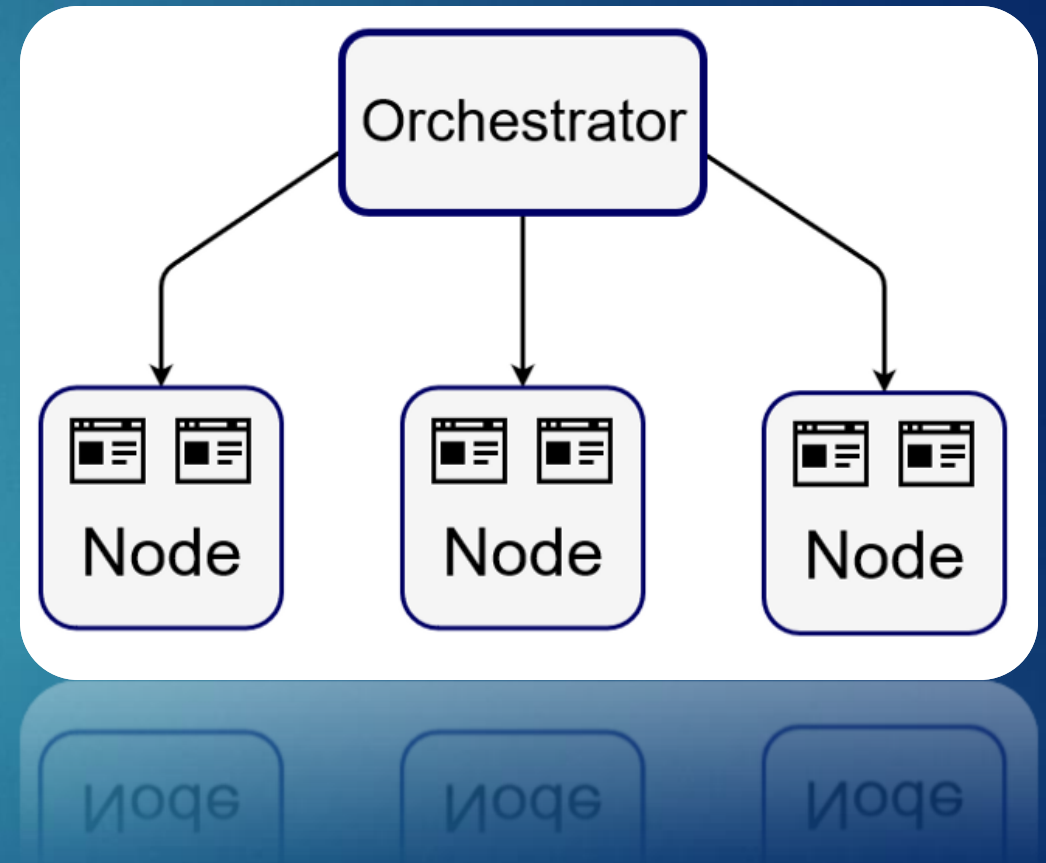
Container Orchestration

- ▶ Container Orchestration is the automation of all aspects of coordinating and managing containers
- ▶ Key features:
 - ▶ Provisioning and deployment of containers
 - ▶ Redundancy and availability of containers
 - ▶ Scaling up or down to spread application load evenly across host infrastructure
 - ▶ External exposure of services running in a container with the outside world
 - ▶ Load balancing between containers
 - ▶ Health monitoring of containers and hosts



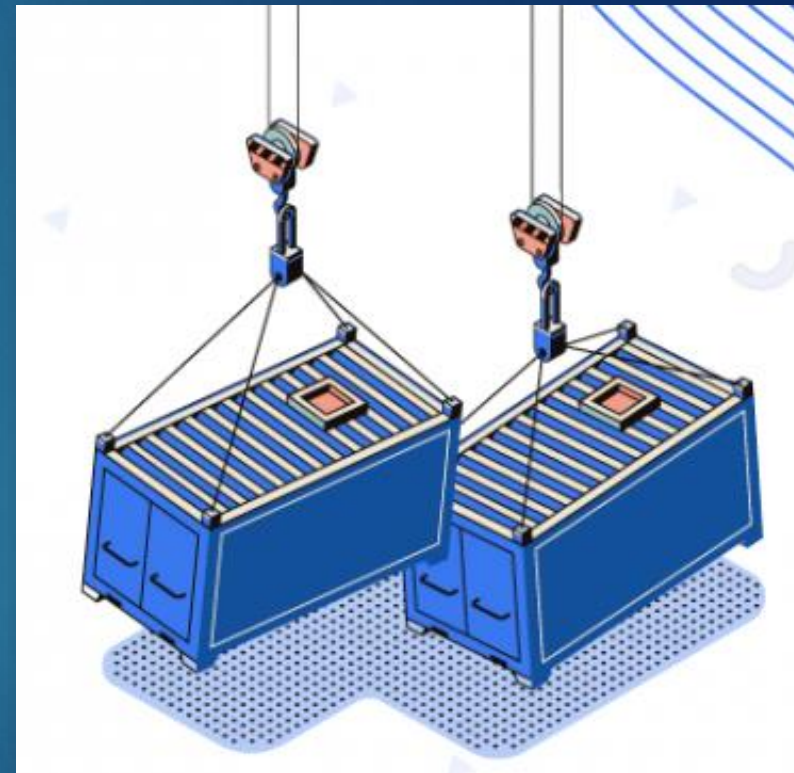
How does Container Orchestration work?

- ▶ You describe the configuration of the application via a YAML or JSON file to the orchestration tool
- ▶ The configuration file specifies information such as the location of the image, networking between containers, storage mounting, etc.
- ▶ The container orchestration tool automatically schedules the deployment of containers to the right host, taking into account any defined requirements or restrictions
- ▶ When the container is placed on the host, the orchestration tool will follow the specifications set in the configuration file to manage the container lifecycle



Benefits of Container Orchestration

- ▶ Scalability
 - ▶ Scale applications with a single command and only scale specific functions without affecting the entire application
- ▶ Simple and fast deployment
 - ▶ Quickly create new containerized applications to address growing traffic
- ▶ Resilience
 - ▶ Container orchestration tools can automatically restart or scale a container or cluster
- ▶ Improved security
 - ▶ Share specific resources without risking internal or external security



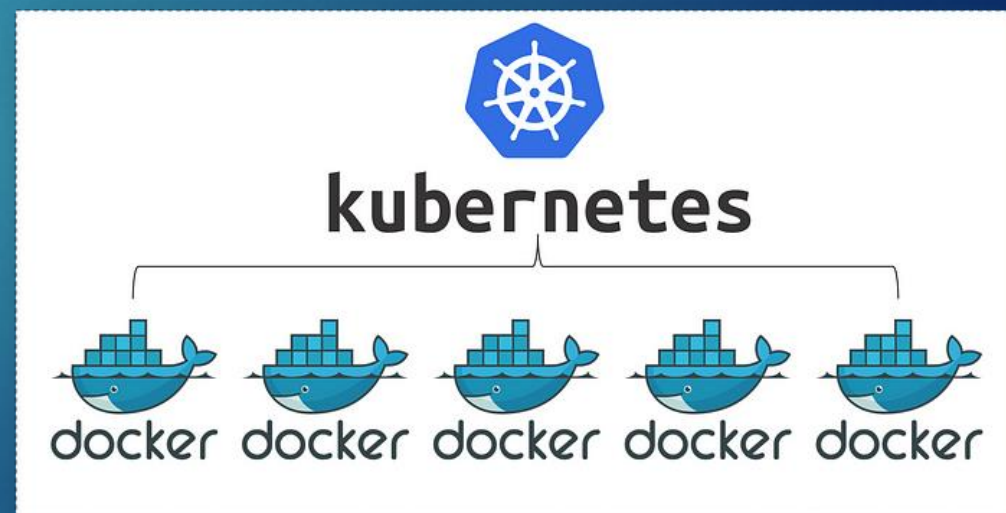
Container Orchestration Tools

- ▶ Kubernetes
 - ▶ “Gold standard” orchestration system for Docker containers. It handles scheduling and manages workloads based on user-defined parameters
- ▶ OpenShift
 - ▶ Comprehensive Kubernetes platform developed by Red Hat that provides enterprises with the tools to build, deploy, and manage containerized applications in both hybrid and multi-cloud environments
- ▶ Docker Swarm
 - ▶ Provides native clustering functionality for Docker containers, which turns a group of Docker engines into a single, virtual Docker engine
- ▶ Google Kubernetes Engine
 - ▶ Built on Kubernetes, lets you run Docker containers on the Google Cloud
- ▶ Amazon Elastic Container Service (ECS)
 - ▶ Supports Docker containers and lets you run applications on a managed cluster of Amazon EC2 instances



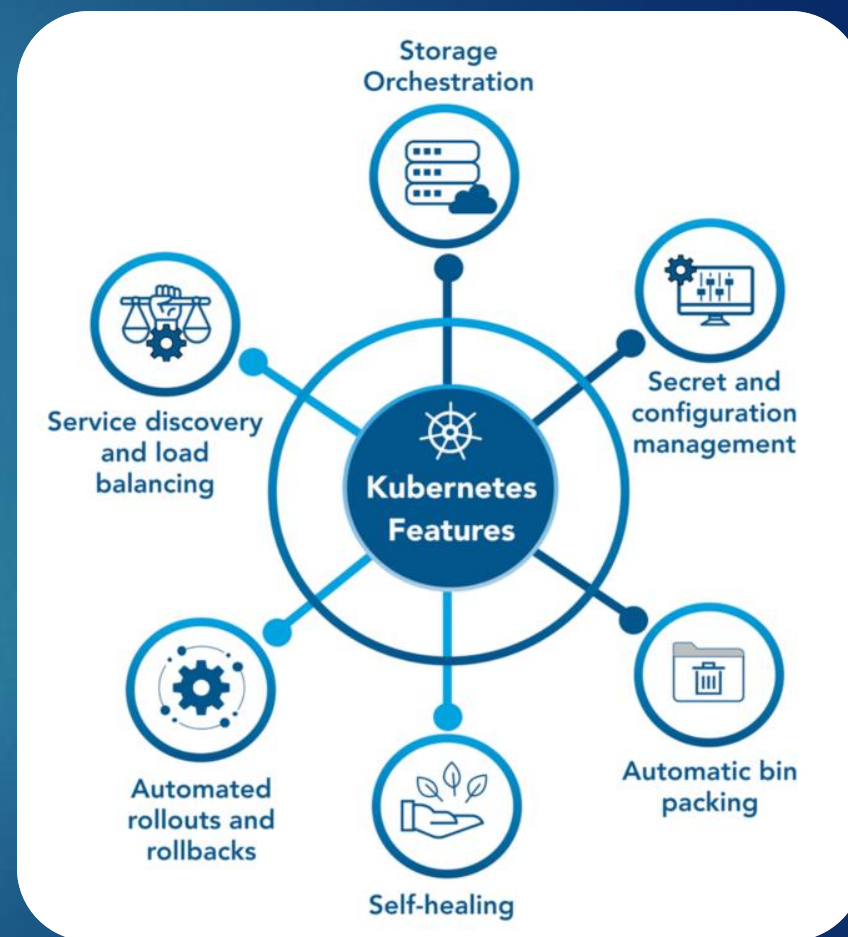
Introducing Kubernetes

- ▶ Kubernetes (aka K8s) is an open-source container orchestration platform for automating deployment, scaling, and management of containerized applications.
- ▶ It abstracts infrastructure management and ensures scalability, high availability, and portability.
- ▶ Designed for complex, distributed applications
- ▶ Originally designed by Google, the project is now maintained by a worldwide community of contributors, and the trademark is held by the Cloud Native Computing Foundation



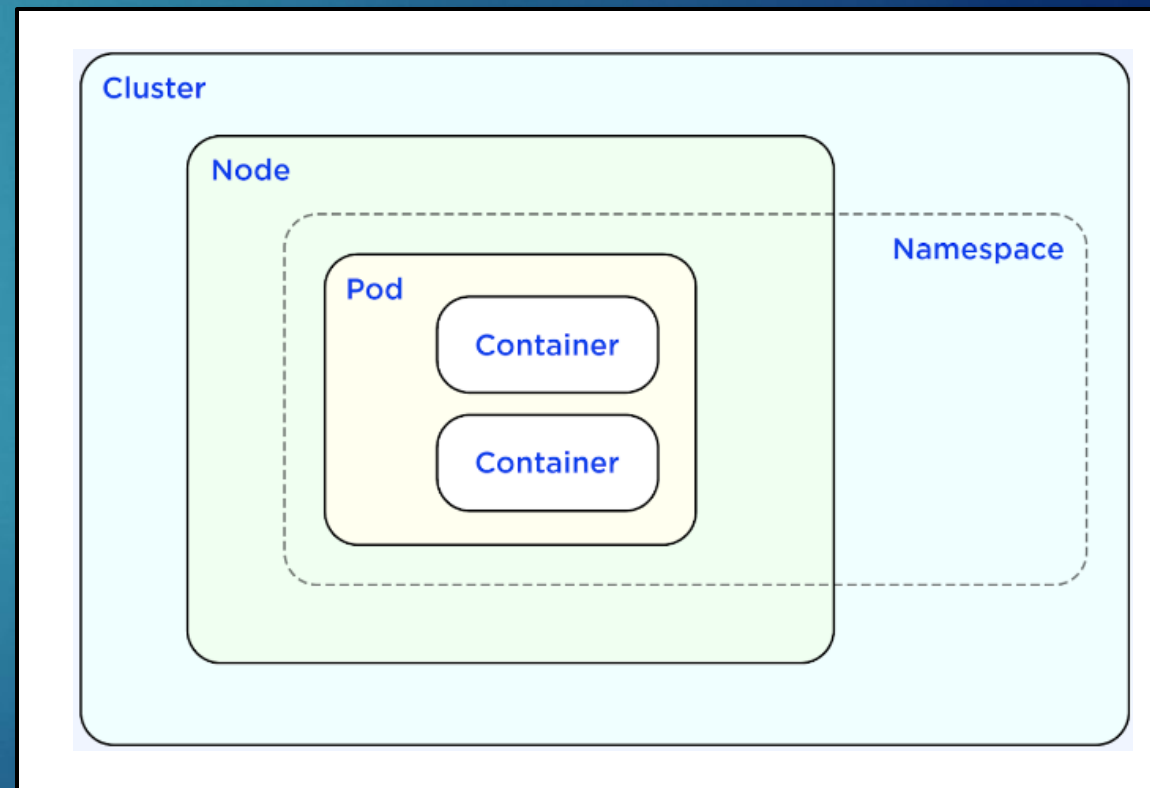
Kubernetes Features

- ▶ **Service Discovery and Load Balancing:** Kubernetes assigns a DNS name and IP to each set of pods and balances traffic across multiple pods during high traffic
- ▶ **Automated Rollouts and Rollbacks:** Kubernetes handles rolling updates to deploy new pods and configurations seamlessly, with rollback functionality if changes fail
- ▶ **Secret and Configuration Management:** Securely stores and updates secrets and configuration data without rebuilding images, reducing the risk of data exposure
- ▶ **Storage Orchestration:** Supports mounting diverse storage types, including local, network, and cloud storage
- ▶ **Automatic Bin Packing:** Efficiently allocates containers based on resource requirements like CPU and RAM to optimize usage
- ▶ **Self-Healing:** Automatically performs health checks, restarts failed containers, and avoids routing traffic to unhealthy pods



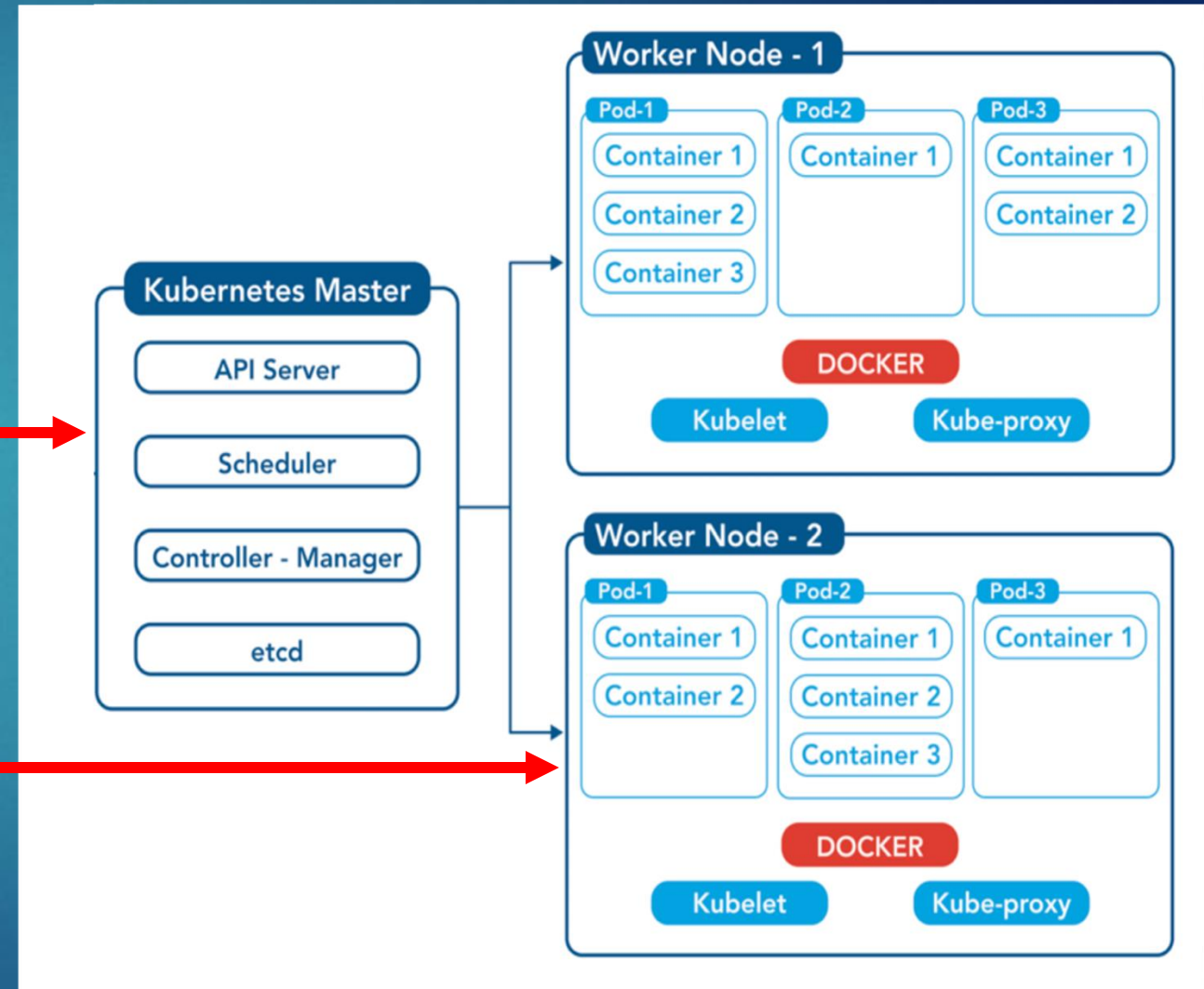
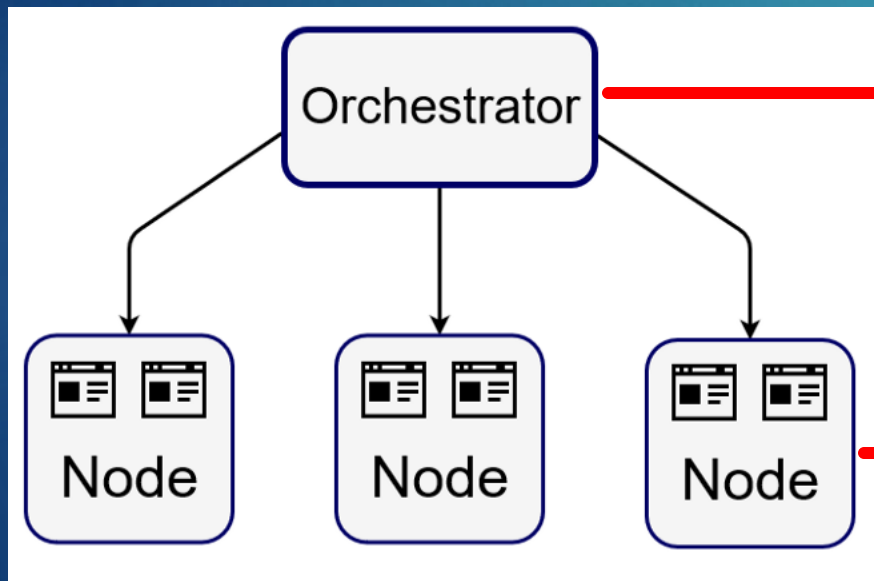
Kubernetes – Core Concepts

- ▶ A **cluster** is group of machines (nodes) running Kubernetes
- ▶ A (worker) **node** is single machine (physical or virtual) in the cluster, responsible for running workloads
- ▶ A **pod** is the smallest deployable unit in Kubernetes, containing one or more containers
- ▶ A **namespace** is Logical partitions within a cluster, isolating resources
 - ▶ These helps manage multiple teams or projects on a single cluster



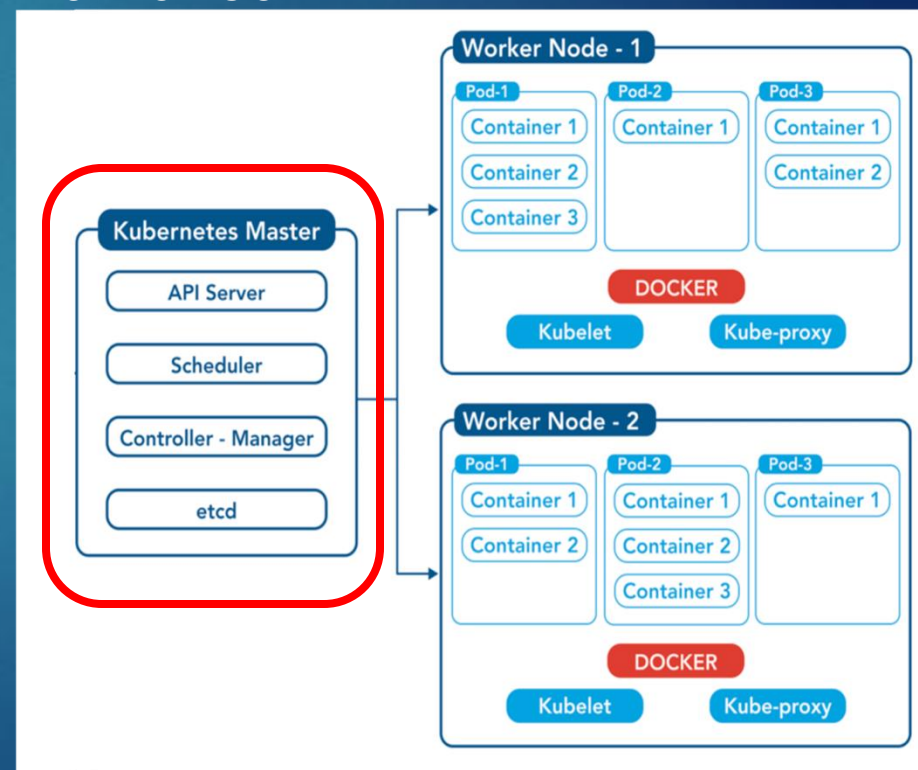
Kubernetes Architecture

15



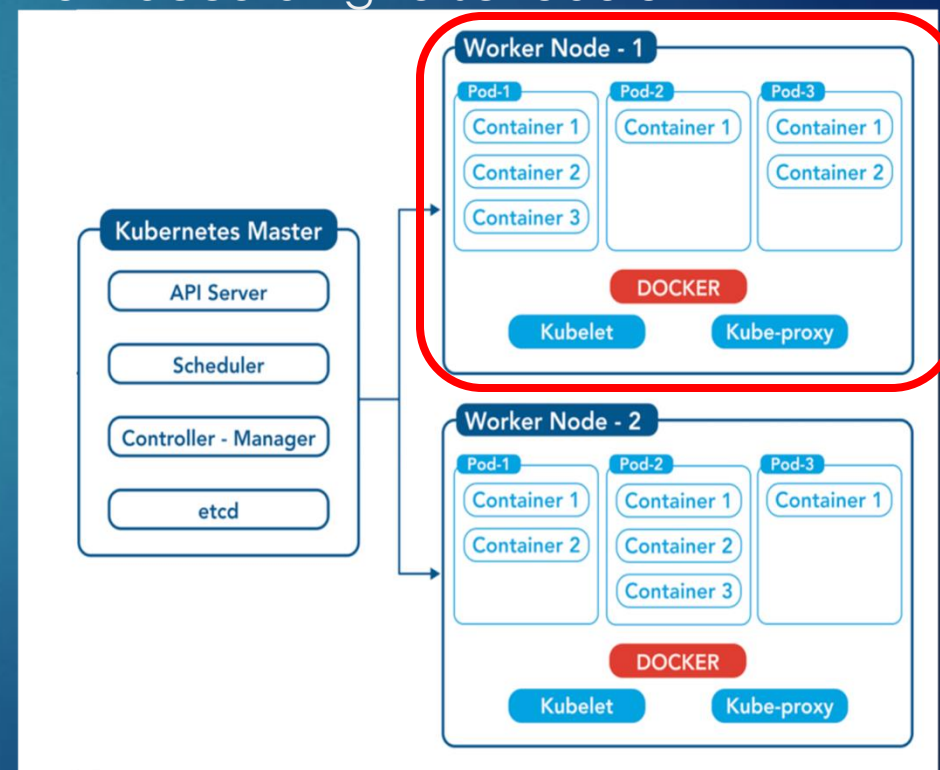
Kubernetes Architecture - Master

- ▶ The Kubernetes Master (aka. Control Plane) a set of components that collectively manage the state of a Kubernetes cluster. There are 4 main components:
- ▶ The **API Server** acts as the central management interface. Validates requests, processes configuration data (e.g., deployments, services), and interacts with other components like the scheduler and etcd to ensure the desired cluster state is maintained
- ▶ **Etcd** is distributed key-value store that serves as Kubernetes' backing store, holding all cluster configuration and state data reliably
- ▶ The **Controller Manager** ensures the desired state of the cluster by running controllers that handle tasks like node monitoring, replication, and maintaining service endpoints
- ▶ The **Scheduler** assigns pods to nodes based on resource availability, constraints, and scheduling policies, ensuring efficient placement and workload balance in the cluster



Kubernetes Architecture – Worker Node

- ▶ Each worker node in the cluster runs several crucial components that enable the execution and management of containers
- ▶ **Kubelet** serves as the cluster's "node agent," supervising each and every node. By communicating with the Kubernetes API server to handle pod lifecycle events like creation, deletion, and updates, one can be sure that containers work according to schedule
- ▶ **Kube-proxy** is set up on every node and enables services in a cluster connect to one another and handles load balancing, finding services, and network routing, ensuring that traffic gets to appropriate pods

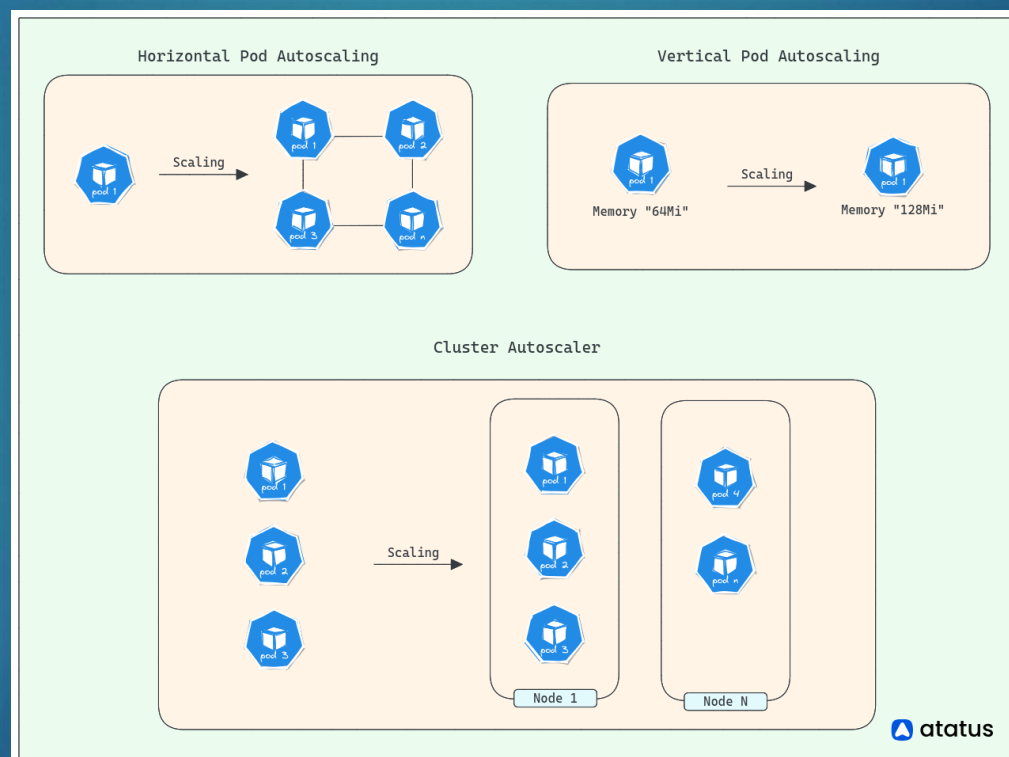


Kubernetes - Auto-scaling

- Kubernetes offers robust Auto-scaling features to ensure optimal resource usage and performance by dynamically adjusting pods and nodes based on workload demands

Horizontal Pod Autoscaling (HPA)

Adjusts the number of pod replicas based on observed metrics like CPU, memory, or custom application metrics. Ideal for scaling applications during high traffic or resource utilization.



Vertical Pod Autoscaling (VPA)

dynamically adjusts the resource requests and limits (CPU/memory) of containers in a pod. Useful for applications with fluctuating resource requirements but consistent replica (pod) counts

Cluster Autoscaler adds or removes nodes (worker instances) in the cluster based on pending or underutilized pods. It ensures the cluster has sufficient capacity to schedule workloads while reducing costs by removing idle nodes.

Kubernetes - Deploying an Application

- ▶ A YAML file is used to define the desired state of resources in Kubernetes, which AWS EKS (Elastic Kubernetes Service) manages
- ▶ They describe Kubernetes objects like Pods, Deployments, Services, and ConfigMaps, which EKS uses to deploy and manage containerized applications
- ▶ Key components of a YAML deployment file:
 - ▶ The **apiVersion** specifies the Kubernetes API version to use for the resource (e.g., apps/v1 for Deployments)
 - ▶ The **kind** indicates the type of Kubernetes resource (e.g., Deployment, Service, Pod)
 - ▶ The **metadata** includes the resource's name, labels, and namespace
 - ▶ The **spec** describes the desired state and configuration of the resource



K8s YAML Template

Kubernetes - Deploying an Application

- The following YAML file nginx-deployment.yaml deploys a simple Nginx container on AWS EKS along with a Horizontal Pod Autoscaler (HPA)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  namespace: default
  labels:
    app: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "256Mi"
```

This is a **deployment** resource

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
  namespace: default
  labels:
    app: nginx
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: LoadBalancer
```

This is a **service** resource

Links service to pods with the label **nginx**

Create an external **load balancer** and exposes port 80 on the service and maps to port 80 in pods

Specifies the image to be pulled from **DockerHub**

Specifies minimum and maximum **resources**

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nginx-hpa
  namespace: default
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx-deployment
  minReplicas: 1
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```

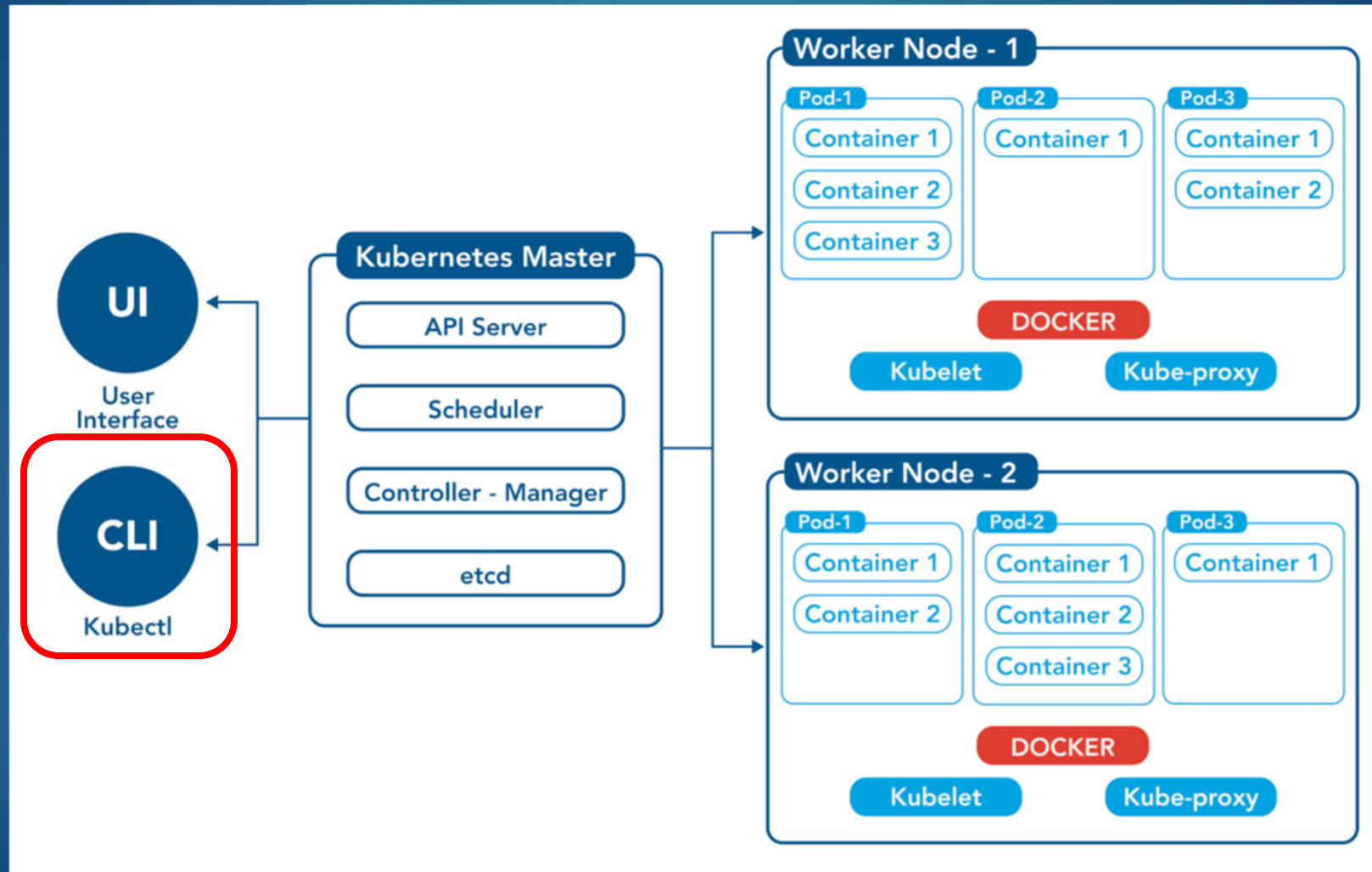
Declares the **HPA** resource type

Declares the minimum and maximum number of **replicas (pod instances)**

Triggers scaling if average **CPU usage exceeds 50%**

Kubernetes Architecture

21



Kubectl

- ▶ **kubectl** is the command-line tool for interacting with Kubernetes clusters. It allows users to deploy, manage, and troubleshoot applications and resources in a Kubernetes environment.
- ▶ Key features:
 - ▶ Cluster Management to connect to and authenticate with Kubernetes clusters and inspect the cluster state, including nodes, namespaces, and workloads
 - ▶ Resource Management to create, update, and delete Kubernetes resources like pods, deployments, services, and secrets
 - ▶ Real-Time Monitoring to view the status of resources and detailed descriptions
 - ▶ Troubleshooting to debug workloads and analyze issues by viewing events and resource configurations



Kubectl - Examples

- Applies the configurations from the nginx-deployment.yaml file to the cluster, creating the resources specified

```
$ kubectl apply -f nginx-deployment.yaml
```

```
deployment.apps/nginx-deployment created
service/nginx-service created
horizontalpodautoscaler.autoscaling/nginx-hpa created
```

- Displays the pods created by the deployment, along with their statuses (e.g., Running).

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-deployment-7c4f4b8d8c-tz2c4	1/1	Running	0	15s

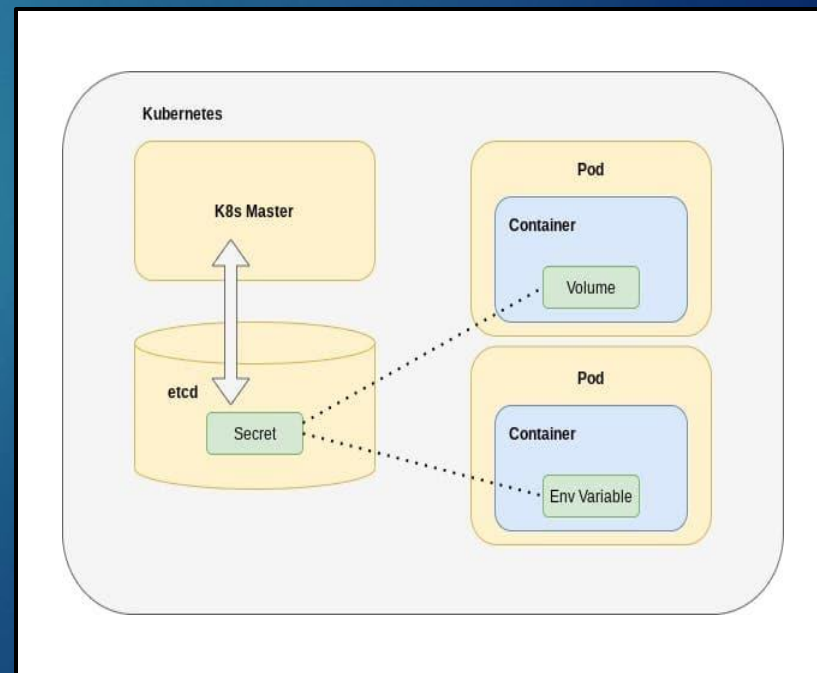
- Lists services, showing the cluster IP and external IP (if available) for accessing nginx

```
$ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
nginx-service	LoadBalancer	10.96.123.45	52.15.123.67	80:30000/TCP	25s

Kubernetes Security with Secrets

- ▶ **Kubernetes Secrets** securely store sensitive information like passwords, API keys, or certificates, separating configuration data from application code and reducing exposure risk
- ▶ Key Features:
 - ▶ **Secure Storage**
 - ▶ Secrets are stored as Kubernetes objects and can be encrypted at rest in **etcd**
 - ▶ They are not hard-coded in application code or images, reducing exposure
 - ▶ **Dynamic Management**
 - ▶ Secrets can be updated independently of the application, ensuring live updates without restarting pods
 - ▶ Containers using the updated secret automatically reflect the changes
 - ▶ **Flexible Usage**
 - ▶ Secrets can be injected into pods as **environment variables**, **volume-mounted files**, or both



Kubernetes Security with Secrets - Example

- ▶ Create a secret with the kubectl command-line tool

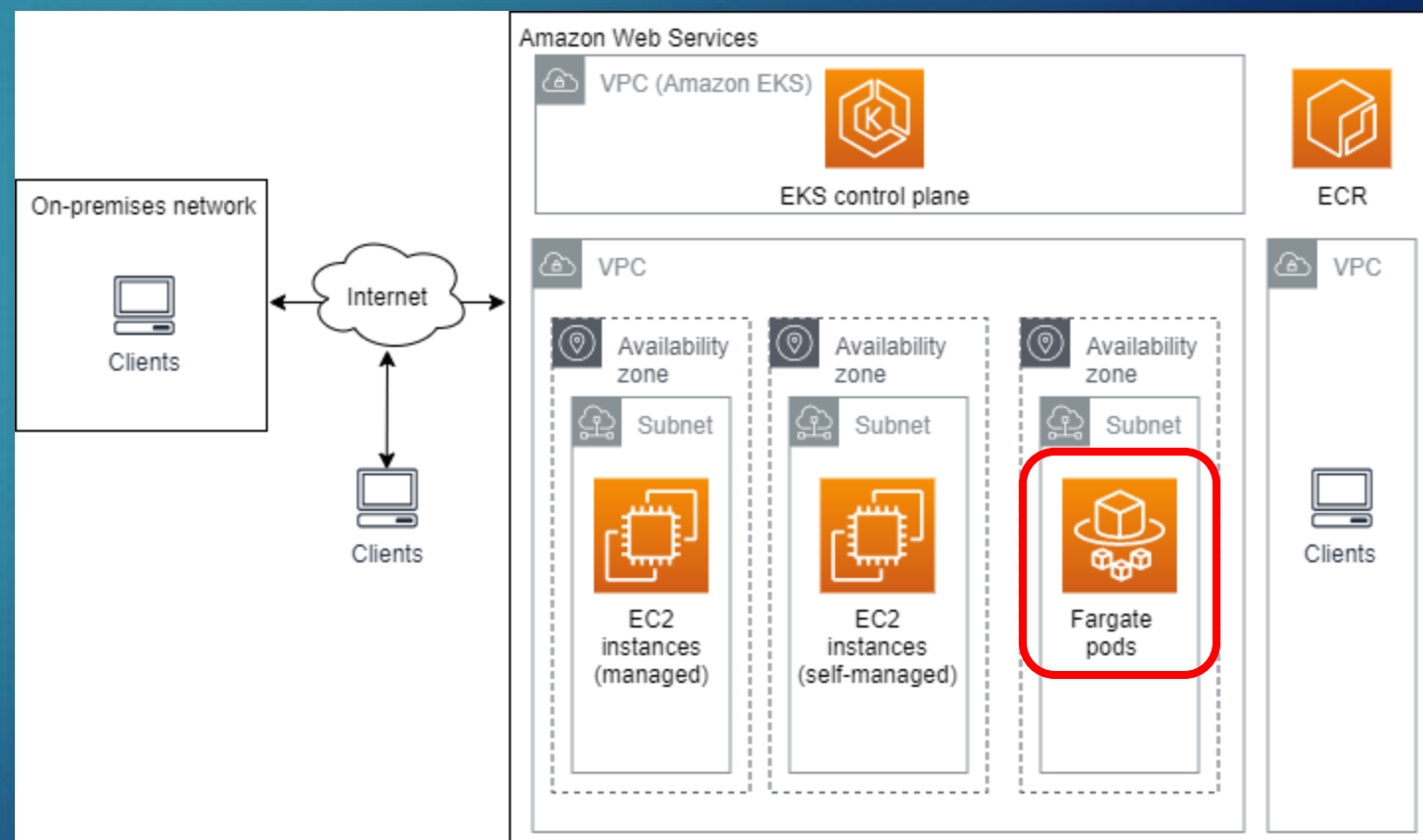
```
$ kubectl create secret generic db-secret --from-literal=db-password=MySecurePassword
```

- ▶ db-secret
 - ▶ The name of the secret
- ▶ --from-literal=db-password=MySecurePassword
 - ▶ Key-value pair where:
 - ▶ db-password is the key
 - ▶ MySecurePassword is the value

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: db-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: db-app
  template:
    metadata:
      labels:
        app: db-app
    spec:
      containers:
        - name: app
          image: nginx
          env:
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: db-secret
                  key: db-password
```

Amazon Elastic Kubernetes Service (EKS)

- ▶ EKS is a fully managed Kubernetes service that simplifies the deployment, management, and scaling of containerized applications
- ▶ It integrates seamlessly with AWS infrastructure and services, providing a secure and scalable Kubernetes environment
- ▶ 100% compatible with standard Kubernetes - runs any standard Kubernetes application
- ▶ Uses same tooling and APIs as self-managed Kubernetes
- ▶ Supports the latest Kubernetes versions

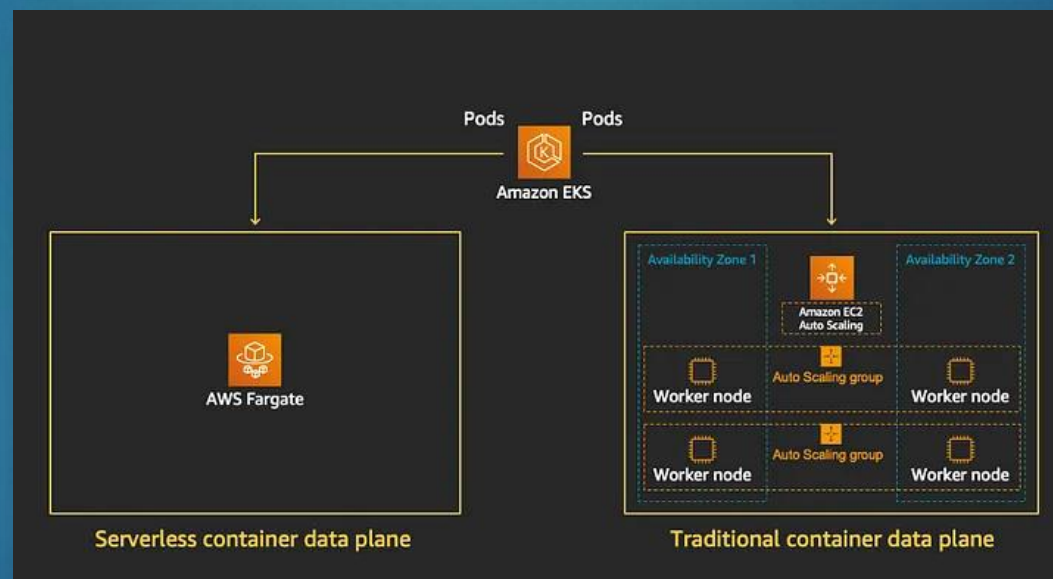


Amazon Elastic Kubernetes Service (EKS)

- **Fargate** is a serverless compute engine which works with ECS and EKS where the management of the underlying VM is performed by AWS and the user only needs to manage the development and deployment of the application

Fargate Ideal For:

- Microservices with variable workloads
- Dev/Test environments
- Teams without infrastructure expertise
- Applications requiring strict workload isolation
- Batch processing jobs



EC2 Ideal For:

- High-performance applications
- Workloads requiring specific instance types
- Stateful applications
- Cost-sensitive production workloads
- Applications requiring full host access

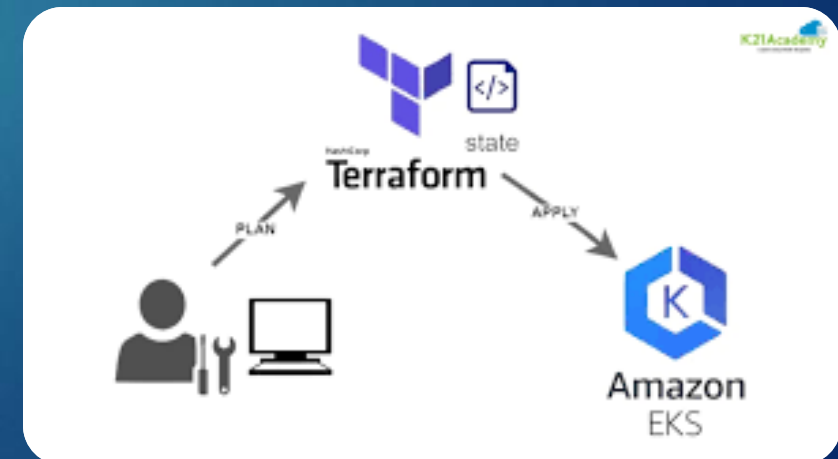
EKS vs. Self-Managed Kubernetes

Feature	EKS	Self-Managed Kubernetes
Control Plane Management	Fully managed by AWS	Self-managed
Maintenance Overhead	Minimal - AWS handles updates/patches	High - Team manages everything
Integration with AWS Services	Native, seamless integration	Requires manual configuration
Cost	Pay for control plane + worker nodes	Pay for infrastructure only
Setup Complexity	Simplified with AWS tools	Complex initial setup
Control/Flexibility	Less control but standardized	Complete control but more responsibility

Using Terraform with EKS

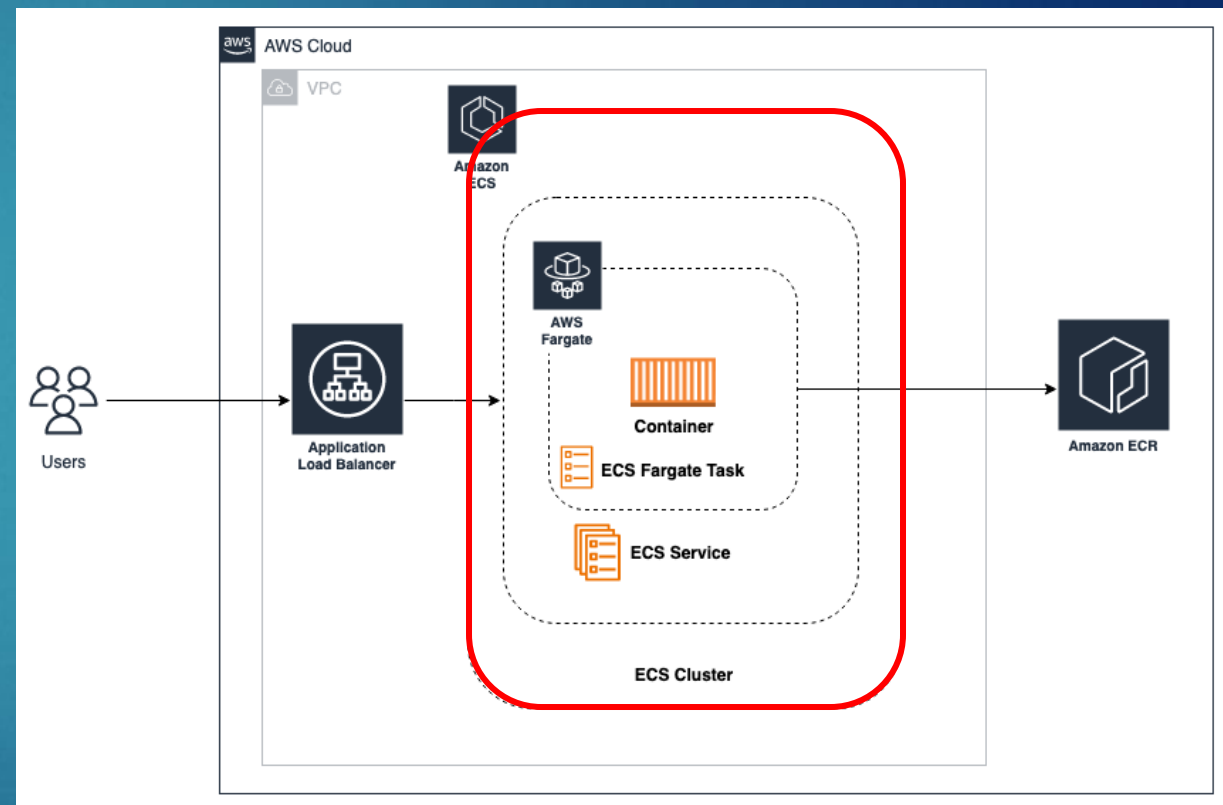
29

- ▶ Setting up an Amazon EKS cluster involves creating and managing numerous interconnected resources. With Terraform, you can streamline this complex process by defining your entire EKS environment in a single, reusable configuration file
- ▶ Other Key Benefits:
 - ▶ EKS requires resources like VPCs, IAM roles, security groups, node groups, and add-ons. Terraform allows you to manage these resources cohesively, avoiding manual steps across the AWS Management Console.
 - ▶ Automate the creation of over 60 resources required for a fully functional EKS setup, ensuring consistent configurations across environments (e.g., development, staging, production)
 - ▶ Terraform automatically resolves dependencies between AWS resources, ensuring that components like VPCs and subnets are available before node groups or services are provisioned
 - ▶ With Terraform modules, you can reuse configurations and easily extend your setup to include additional resources, such as monitoring or auto-scaling policies, with minimal effort



Amazon Elastic Container Service (ECS)

- ▶ **ECS** is a fully managed container orchestration service to enable users to run, scale, and manage Docker containers on a cluster of virtual machines. Features include:
- ▶ Cluster Management
 - ▶ Supports clusters of Amazon EC2 instances or AWS Fargate, which is a serverless compute engine for containers
- ▶ Task Definition
 - ▶ Defines the containers to run as part of a task, including image, CPU, memory requirements and networking configuration
- ▶ Services
 - ▶ Ensures a specified number of tasks are continuously running and can be automatically scaled and managed
- ▶ Auto-Scaling
 - ▶ ECS supports Auto-scaling for both the containerized applications and the underlying EC2 instances, ensuring that applications can dynamically adjust to varying workloads



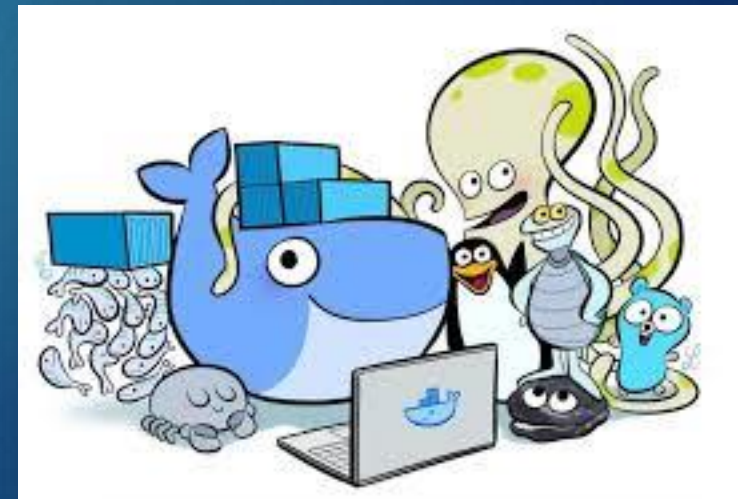
ECS vs. EKS

31

	 ECS	 EKS
Open Source	AWS proprietary	Yes, Kubernetes
Container Unit	Task	Pod
Effort	Easy	Harder - AWS plus Kubernetes knowledge
Security IAM	Managed Service	Add-on software and configuration
Security ENI	Per Task serving one container	Per Pod which can serve multiple containers
VM container limit	120	750 pods
Multi-cloud integration	AWS proprietary	Yes, Kubernetes

Summary

- ▶ Containers require just enough of an operating system, supporting programs and libraries, and system resources to run a specific program(s)
 - ▶ From Developer's perspective - build once, run anywhere
 - ▶ Eliminate inconsistencies between development, test, production, and customer environments
- ▶ VMs are still a better choice for running apps that require all the operating system's resources and functionality
- ▶ Containers are more lightweight vs. VMs as they can start in seconds, or even milliseconds in some cases
- ▶ From a performance perspective, containers are a much better execution foundation for microservices architecture



WordPress on EKS

- ▶ In this activity, you will be building an Elastic Kubernetes Service (EKS) cluster using Terraform and simulating both fault tolerance and Auto-scaling
- ▶ This will be using the 2 images you have created and stored in ECR so make sure this is successfully working before starting this activity
- ▶ This activity can be accessed at [Assignments > Activity – WordPress on EKS](#)
- ▶ There will be a slight charge for using EKS, which is not covered under the free tier
- ▶ Note: **Allocate at least 90 minutes** to do this activity, as the setup and teardown will take several minutes to run

